

Agentic Architect Challenge: Architecture Brief

Prepared by: Adam Danish Bin Muhamad Azizi

Part 1: Customer support email agent

The system identifies the customer and counts their support contacts in the last seven days. Before any AI draft, it sends emails mentioning data loss, a service outage, or a security breach to a human. It also flags a customer's fourth contact within seven days and sends it to a human.

For other emails, an AI agent classifies the topic, searches approved FAQs and PDFs, and drafts a reply using the information it finds. Refund answers must use the current approved refund policy. If the policy is unclear, a search fails, or a draft contains an unsupported fact, a human handles the case. A support worker reviews, edits, and sends every accepted draft. This takes more time but lowers the risk of sending an incorrect answer. The proposed system would log routing reasons, document sources, errors, and response times.

Part 2: Web summarizer

The script downloads a webpage and uses Trafilatura to extract its main text. If it finds fewer than 30 words, Playwright opens the page in Chromium and extraction is tried again. This helps with pages whose text appears after JavaScript runs. The browser step is slower and can also run unnecessarily on a genuinely short page.

For long pages, the script splits the text into 3,000-word chunks. Gemini summarizes each chunk, then combines the results. A final word-count check keeps the output within 100 words. Many chunks require many API calls, so a page may hit a rate limit. Extraction failures stop without a summary; Gemini 429 and 503 errors show a clear message. The word limit controls length but does not guarantee factual accuracy.

Part 3: Document-based event assistant

The assistant reads the supplied one-page visitor guide and uses its text as the source for event answers. Its chat session remembers details the user shares, such as their name. Gemini can call a calculator for ticket-total questions; it answers ordinary fact questions without that tool. When the guide lacks an answer, the assistant is instructed to say the document does not say.

Using the whole document keeps the design simple, but memory ends when the conversation ends. Ticket prices are fixed in the calculator code and must change if the guide's prices change. An unreadable PDF stops the app, and API errors are shown to the user.